

Experiment 5 | Lab Report

BMP280 Sensor Data Reading for Pressure, Temperature and Altitude

Reading a barometric sensor over I²C on a Raspberry Pi, and publishing it from an ESP32 as a live web dashboard

Submitted by	Chavan Atharva Sunil
Roll Number	240003021
Date of Experiment	3 September 2026 (ESP32) and 10 September 2026 (Raspberry Pi)
Boards Used	Raspberry Pi 4 (I ² C bus 1, GPIO2/GPIO3) and ESP32 DevKit (I ² C on GPIO21/GPIO22)

1 Aim

To read **atmospheric pressure** and **temperature** from a **BMP280** barometric sensor over the I²C bus, and to derive the **altitude** from the pressure. The sensor is read first from a **Raspberry Pi**, printing a new set of values to the terminal every second, and then from an **ESP32**, which hosts its own Wi-Fi access point and serves a web page that shows the three quantities live in a browser.

Objectives

- To connect a BMP280 breakout module to a Raspberry Pi and to an ESP32 over the two-wire I²C bus and identify it at its address 0x76.
- To read the compensated pressure and temperature from the sensor with a driver library on each board, and to convert the pressure into an altitude with the barometric formula.
- To present the readings in two ways: as a continuously updating terminal printout on the Pi, and as a web dashboard served by the ESP32 to any device that joins its Wi-Fi network.
- To compare the pressure and pressure-altitude recorded on two different days at the same place, and to understand what the “altitude” of a barometer actually measures.

Brief Theory

The BMP280 is a digital barometric pressure sensor from Bosch. A piezo-resistive sensing element and a temperature sensor feed a 24-bit ADC; the chip’s own factory calibration constants, stored in its non-volatile memory, are read out by the driver and used to *compensate* the raw values into a temperature in °C and a pressure in Pa. It measures 300–1100 hPa with an absolute accuracy of about ±1 hPa and a *relative* accuracy of ±0.12 hPa, which corresponds to a height change of about one metre — fine enough to notice the sensor being raised from a desk to head height.

Pressure falls with height in a known way, so a barometer can be turned into an altimeter. With the standard-atmosphere sea-level pressure $p_0 = 1013.25$ hPa the height is

$$h = 44330 \left[1 - \left(\frac{p}{p_0} \right)^{1/5.255} \right] \text{ m,}$$

which both driver libraries use. The result is a *pressure altitude*: it is only the true altitude if the sea-level pressure on the day really is p_0 . Weather moves p_0 by tens of hectopascals, and every hectopascal shifts the computed height by roughly 8.5 m near sea level — a point that the two runs in this report illustrate directly.

The I²C bus

Unlike the DHT11 of Experiment 4, which needed a bit-banged one-wire protocol with microsecond timing, the BMP280 speaks the standard I²C protocol, which both boards support in hardware. Two lines are shared by every device on the

bus: SDA (data) and SCL (clock). Both are open-drain and held HIGH by pull-up resistors — here the ones on the breakout module — and any device can only pull them LOW. The master (the Pi or the ESP32) generates the clock and starts each transaction by sending the 7-bit address of the device it wants to talk to; the BMP280 answers on 0x76 when its SDO pin is tied to ground (0x77 if tied to VCC). Reading the sensor is then a matter of writing a register address and reading back the bytes, which the library handles.

Same pins, opposite setting. On the Raspberry Pi the I²C bus uses GPIO2 (SDA) and GPIO3 (SCL) — the same GPIO2 that Experiment 4 drove as a plain output. For the DHT11 the I²C interface had to be *disabled* in raspi-config to free the pin; for the BMP280 it must be *enabled*, after which the sensor appears in `i2cdetect -y 1` at address 0x76. On the ESP32 the default I²C pins are GPIO21 (SDA) and GPIO22 (SCL).

2 Materials Required

#	Component	Qty.	Purpose in the setup
1	Raspberry Pi 4 (in case, with SD card, OS installed)	1	Part A host; I ² C master, prints readings to the terminal
2	ESP32 DevKit (30-pin)	1	Part B host; I ² C master, Wi-Fi access point and web server
3	BMP280 breakout module (six-pin: VCC, GND, SCL, SDA, CSB, SDO)	1	Measures pressure and temperature; shared between the two parts
4	Female–female jumper wires	4	VCC, GND, SDA and SCL between header and module
5	Micro-USB cable and laptop with Arduino IDE and Wi-Fi	1	Programming the ESP32 and viewing the dashboard
6	USB-C power supply, monitor, keyboard and mouse	1	Running and observing the Raspberry Pi
7	Software: adafruit-circuitpython-bmp280 (Pi); Adafruit BMP280 library and ESP32 core (Arduino IDE)	—	Sensor drivers on the two boards

3 Methodology

The same breakout module was used on both boards, wired the same way each time: four jumper wires for VCC (3.3 V), GND, SDA and SCL, with the module's CSB and SDO pins left as the board sets them (I²C mode, address 0x76). No breadboard or extra components were needed because the pull-up resistors are on the module.

3.1 Connections

Module pin	Raspberry Pi 4 (Part A)	ESP32 (Part B)	Function
VCC	Pin 1 — 3.3 V	3V3	Supply (the BMP280 is a 3.3 V part)
GND	Pin 6 — GND	GND	Ground
SDA	Pin 3 — GPIO2 (board.SDA)	GPIO21 (Wire default)	Serial data, bidirectional
SCL	Pin 5 — GPIO3 (board.SCL)	GPIO22 (Wire default)	Serial clock from the master
CSB, SDO	not connected	not connected	Module defaults: I ² C mode, address 0x76

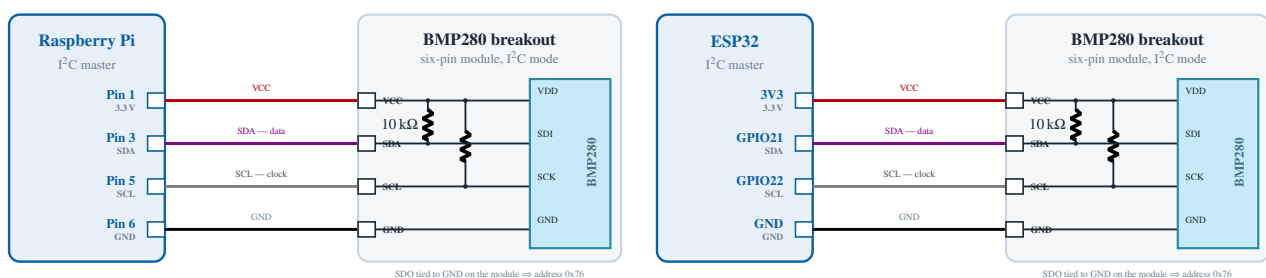


Figure 1: Circuit schematics for the two parts. The wiring is identical apart from the pin names: four wires, with the bus pull-ups and the address-setting SDO connection already on the breakout module.

3.2 Part A — Raspberry Pi, readings on the terminal

With I²C enabled, the sensor was confirmed on the bus with `i2cdetect -y 1` and the Adafruit CircuitPython driver was installed:

Listing 1: Preparing the Raspberry Pi.

```
1 sudo raspi-config      # Interface Options -> I2C -> Enable
2 sudo apt install i2c-tools
3 i2cdetect -y 1        # the BMP280 shows up at 0x76
4 pip3 install adafruit-circuitpython-bmp280
```

The program (Listing 2) opens the bus, creates the sensor object at 0x76, sets the sea-level reference to 1013.25 hPa, and then loops once a second reading the three properties temperature, pressure and altitude and printing them.

Listing 2: Part A — pressure.py on the Raspberry Pi: temperature, pressure and altitude every second.

```
1 import time
2 import board
3 import busio
4 import adafruit_bmp280
5
6 # Initialize I2C
7 i2c = busio.I2C(board.SCL, board.SDA)
8
9 # BMP280 sensor
10 bmp280 = adafruit_bmp280.Adafruit_BMP280_I2C(
11     i2c,
12     address=0x76
13 )
14
15 # Reference pressure at your starting location
16 # Change this if you want better altitude accuracy
17 bmp280.sea_level_pressure = 1013.25
18
19 while True:
20     temperature = bmp280.temperature
21     pressure = bmp280.pressure
22     altitude = bmp280.altitude
23
24     print("-----")
25     print(f"Temperature : {temperature:.2f} °C")
26     print(f"Pressure    : {pressure:.2f} hPa")
27     print(f"Altitude     : {altitude:.2f} m")
28
29     time.sleep(1)
```

3.3 Part B — ESP32, readings on a web dashboard

On the ESP32 the sensor is read with the Adafruit BMP280 Arduino library, but instead of printing to a serial monitor the board publishes the values over Wi-Fi. The sketch (Listing 3) starts the ESP32 as a Wi-Fi **access point** — it creates its own network, so no router is needed — and runs a small web server on it. The server answers two requests: `/` returns the dashboard page, and `/data` returns the current readings as a short JSON object. A few lines of JavaScript in the page fetch `/data` every second and write the numbers into three cards, so a laptop or phone that joins the ESP32's network and opens `http://192.168.4.1` sees the readings update live.

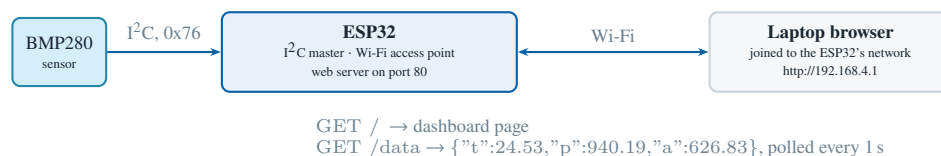


Figure 2: Part B. The ESP32 reads the sensor over I²C and serves the values over its own Wi-Fi network; the page in the browser polls the JSON endpoint once a second.

Listing 3: Part B — ESP32 sketch: BMP280 over I²C, served as a live web dashboard from the board's own access point.

```

1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <Wire.h>
4  #include <Adafruit_BMP280.h>
5
6  const char *AP_SSID = "ESP32-Sensor";    // the ESP32 creates this Wi-Fi network
7  const char *AP_PASS = "12345678";
8  const float SEA_LEVEL_HPA = 1013.25;    // reference for the altitude calculation
9
10 Adafruit_BMP280 bmp;                    // I2C, default pins SDA = 21, SCL = 22
11 WebServer server(80);
12
13 // The dashboard page. Its script asks the ESP32 for fresh values once per second.
14 const char PAGE[] PROGMEM = R"html(
15 <!DOCTYPE html><html><head><meta charset="utf-8">
16 <meta name="viewport" content="width=device-width,initial-scale=1">
17 <title>ESP32 Sensor</title>
18 <style>
19 body{font-family:sans-serif;background:#1c2738;color:#ddd;text-align:center}
20 h1{color:#4fc3f7} #st{color:#69e069}
21 .card{background:#2b3a50;border-radius:10px;width:230px;margin:14px auto;padding:10px}
22 .v{color:#69e069;font-size:30px;font-weight:bold} .u{font-size:12px;color:#aaa}
23 </style></head><body>
24 <h1>ESP32 Sensor Monitor</h1><div id="st">&#9679; ESP32 Connected</div>
25 <div class="card">Temperature<div class="v" id="t">--</div><div class="u">&deg;C</div></div>
26 <div class="card">Pressure<div class="v" id="p">--</div><div class="u">hPa</div></div>
27 <div class="card">Altitude<div class="v" id="a">--</div><div class="u">meters</div></div>
28 <script>
29 setInterval(async () => {
30   try {
31     const d = await (await fetch('/data')).json();
32     t.textContent = d.t.toFixed(2);
33     p.textContent = d.p.toFixed(2);
34     a.textContent = d.a.toFixed(2);
35     st.textContent = '\u25CF ESP32 Connected'; st.style.color = '#69e069';
36   } catch (e) {
37     st.textContent = '\u25CF ESP32 Disconnected'; st.style.color = '#ff7070';
38   }
39 }, 1000);
40 </script></body></html>)html";
41
42 void handleRoot() { server.send_P(200, "text/html", PAGE); }
43
44 void handleData() {                                // the three readings as one JSON object
45   float t = bmp.readTemperature();                // degrees Celsius
46   float p = bmp.readPressure() / 100.0F;          // Pa -> hPa
47   float a = bmp.readAltitude(SEA_LEVEL_HPA);      // metres
48   String json = "{ \"t\": "; json += String(t, 2);
49   json += " \"p\": "; json += String(p, 2);
50   json += " \"a\": "; json += String(a, 2); json += " }";
51   server.send(200, "application/json", json);
52 }
53
54 void setup() {
55   Serial.begin(115200);
56   Wire.begin(21, 22);                            // SDA, SCL
57   if (!bmp.begin(0x76)) {
58     Serial.println("BMP280 not found - check the wiring and the address");
59     while (true) delay(10);
60   }
61   WiFi.softAP(AP_SSID, AP_PASS);                // start the access point
62   Serial.print("Dashboard at http://");
63   Serial.println(WiFi.softAPIP());                // 192.168.4.1
64   server.on("/", handleRoot);
65   server.on("/data", handleData);
66   server.begin();
67 }
68
69 void loop() { server.handleClient(); }

```

3.4 Procedure

- Part A.** With the Pi off, the module's VCC, GND, SDA and SCL were wired to header pins 1, 6, 3 and 5. I²C was enabled in raspi-config, and i2cdetect confirmed a device at 0x76.
- The driver was installed and Listing 2 was saved as pressure.py and run with python3 pressure.py. The terminal printed temperature, pressure and altitude once a second; the run was recorded on video.
- Part B.** The same module was moved to the ESP32: VCC to 3V3, GND to GND, SDA to GPIO21, SCL to GPIO22. In the Arduino IDE the ESP32 board package and the Adafruit BMP280 library (with its Unified Sensor and BusIO dependencies) were installed and Listing 3 was uploaded.

- The Serial Monitor showed the access-point address, 192.168.4.1. The laptop's Wi-Fi was switched to the ESP32's network and the address opened in a browser; the dashboard appeared with the "ESP32 Connected" indicator and the three values updating every second. This was also recorded.
- The readings of both runs were transcribed from the recordings, and the altitude values were checked against the barometric formula.

4 Result and Output

Both boards read the BMP280 correctly over I²C. On the **Raspberry Pi** the terminal printed a new set of values every second — 14 consecutive readings were recorded, with the pressure steady at 929.1–929.4 hPa, the altitude at 718.6–720.4 m, and the temperature easing from 31.4 to 30.1 °C. On the **ESP32** the web dashboard at 192.168.4.1 updated live, showing 24.5–24.6 °C, 940.1–940.2 hPa and 626.6–627.6 m. Every altitude value agrees with the barometric formula applied to the pressure beside it.

Observed outputs — Part A, Raspberry Pi terminal

#	Temp. (°C)	Pressure (hPa)	Altitude (m)	#	Temp. (°C)	Pressure (hPa)	Altitude (m)
1	31.39	929.12	720.35	8	30.64	929.31	719.04
2	31.27	929.28	719.22	9	30.51	929.18	720.06
3	31.17	929.28	719.00	10	30.47	929.21	719.04
4	31.04	929.15	719.88	11	30.36	929.30	718.87
5	30.96	929.27	718.94	12	30.28	929.29	718.59
6	30.82	929.41	719.28	13	30.19	929.29	718.94
7	30.74	929.27	718.90	14	30.09	929.27	718.90

Readings are one second apart, transcribed from the recording of the terminal (Figure 4). Mean pressure 929.27 hPa with a spread of only ± 0.15 hPa; the altitude scatters by about ± 1 m around 719 m accordingly.

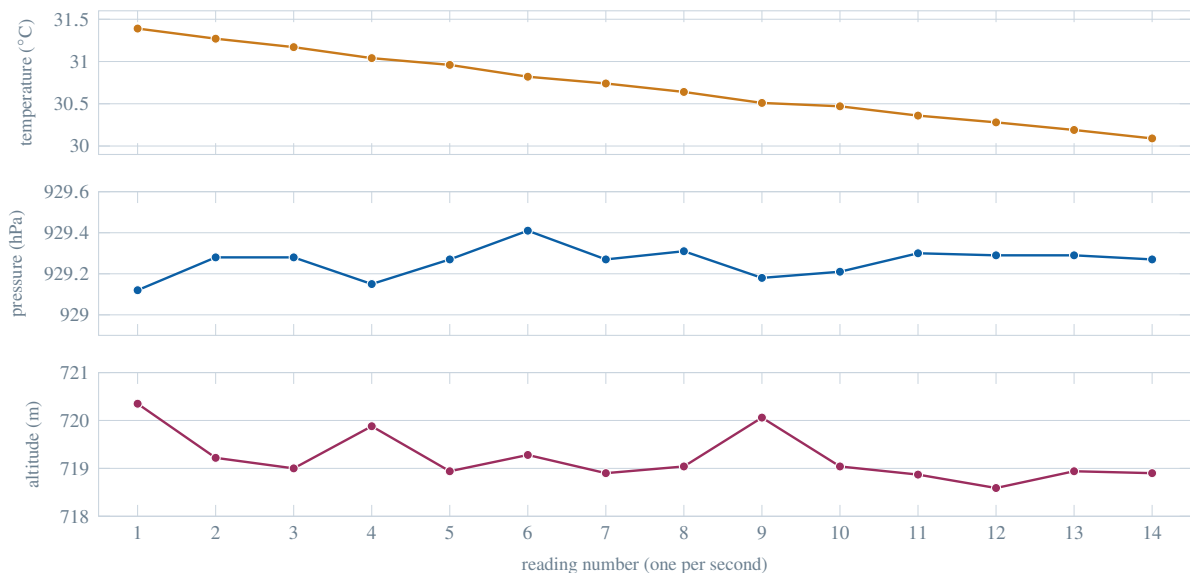


Figure 3: The 14 readings of the Raspberry Pi run. The temperature falls steadily as the module cools after being handled; the pressure is flat to within ± 0.15 hPa, and the altitude, computed from it, jitters by about a metre around 719 m.

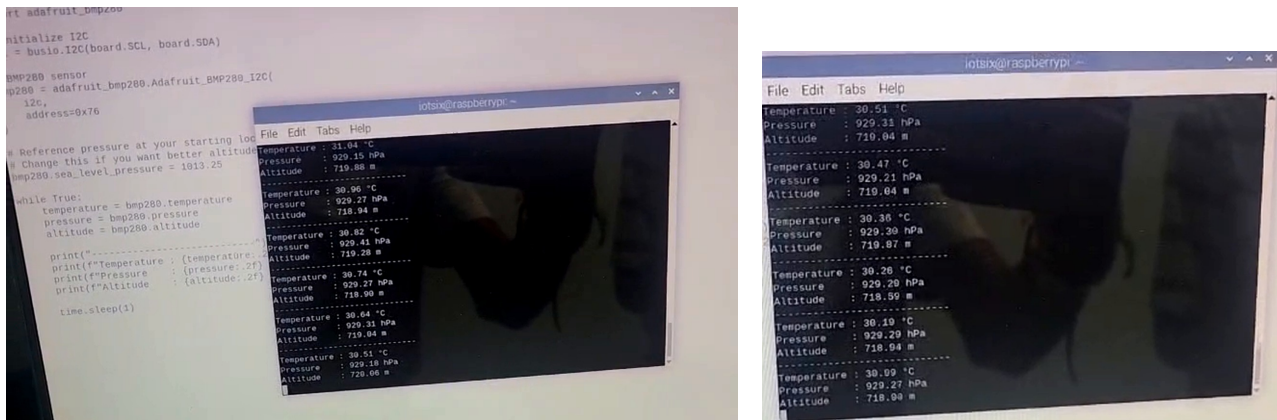


Figure 4: Part A on screen. *Left:* pressure.py open in the editor behind the terminal, which is printing the readings; *right:* a later frame with the last readings of the run. Each block is one second's temperature, pressure and altitude.

Observed outputs — Part B, ESP32 dashboard

Clip time	Temp. (°C)	Pressure (hPa)	Altitude (m)
0 s	24.53	940.19	626.83
1 s	24.53	940.18	626.97
2 s	24.54	940.22	626.57
6 s	24.54	940.20	626.76
8 s	24.54	940.15	627.16
10 s	24.56	940.14	627.31
12 s	24.57	940.15	627.19
16 s	24.58	940.19	626.82
clip 2, 7 s	24.60	940.10	627.62
clip 2, 8 s	24.60	940.13	627.34

Checking the altitude

For the first dashboard value,
 $p = 940.19 \text{ hPa}$:

$$h = 44330 \left[1 - \left(\frac{940.19}{1013.25} \right)^{1/5.255} \right] = 626.8 \text{ m,}$$

exactly the 626.83 m shown. The same check on the Pi's 929.27 hPa gives 724 m, within a few metres of the printed 719 m — the Pi library takes a fresh pressure sample for the altitude, so the two lines are not from the identical measurement.

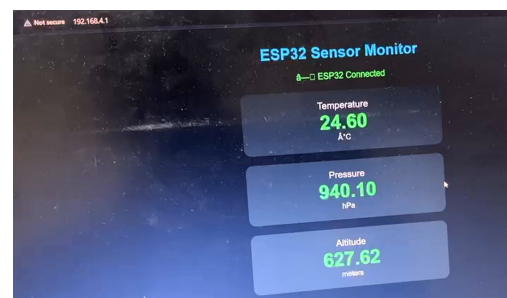
Values read from the dashboard in the two recorded clips. The temperature is six degrees lower than in Part A because this run took place in the air-conditioned lab a week earlier with the module untouched on the desk, and the pressure is 11 hPa higher — a different day's weather — which the formula turns into an “altitude” 92 m lower for the same room.



first clip, $t = 0$



first clip, $t = 9 \text{ s}$



second clip — address bar shows 192.168.4.1

Figure 5: Part B. The dashboard served by the ESP32, viewed in the laptop's browser at the access point's address 192.168.4.1: the “ESP32 Connected” indicator and three cards whose values change every second. (The stray Å before the degree sign in the recording is a character-encoding slip in the page as run that day; see the Discussion.)

The setups

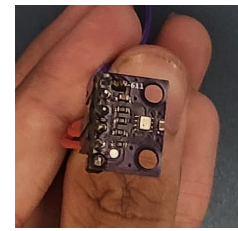
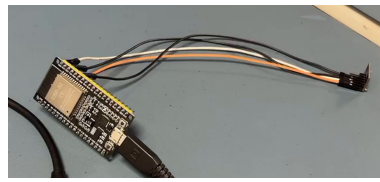
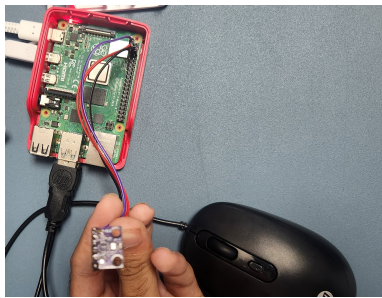


Figure 6: Left: Part A — the Raspberry Pi 4 with the BMP280 module on four jumper wires. Centre: Part B — the ESP32 DevKit with the same module. Right: the six-pin breakout with the BMP280 (the small metal package).

Video demonstration and code

Google Drive link

The demonstration videos, this report and the complete source code are available in the following shared folder:

https://drive.google.com/drive/folders/11TP-NX_EF_IpxHIAUYmG8YWPQfst2ae2?usp=sharing

Access has been set to *anyone with the link can view*.

Discussion

- **What the altitude really is.** The same room gave 719 m on one day and 627 m on another, and Indore actually lies at roughly 550 m. None of these is a sensor fault: the formula assumes a sea-level pressure of 1013.25 hPa, and on both days — in the monsoon season — it was lower, by different amounts. A barometric altimeter must therefore be given the *actual* sea-level pressure (the “QNH” from a weather station) through `sea_level_pressure`, after which it reads true altitude; or it is used for *relative* height, where the reference cancels and the ± 0.12 hPa resolution gives about one metre.
- **Noise and averaging.** Successive pressure readings scatter by ± 0.15 hPa, which is ± 1.3 m of altitude — visible as the jitter in Figure 3 and in the dashboard’s last digits. The BMP280’s oversampling and on-chip IIR filter, or a software average, smooth it.
- **Temperature is the chip’s, not the room’s.** The BMP280 measures its own die temperature, mainly to compensate the pressure. It reads high when the module is warm: 31 °C and falling in Part A right after the module had been held, 24.5 °C and steady in Part B on the desk in the air-conditioned lab.
- **Two ways to deliver the data.** The Pi prints to its own terminal, which needs a screen; the ESP32 has no screen but has Wi-Fi, so it serves the data as a web page. As an access point it works anywhere, without a router — the laptop joins the ESP32’s network and opens 192.168.4.1. Polling a small JSON endpoint once a second is the same pattern an IoT node uses to report to a server.
- **The stray character.** The recorded dashboard shows “Â°C”: the page sent a raw degree sign without declaring its encoding, so the browser read its two UTF-8 bytes as two Latin-1 characters; Listing 3 declares the charset.
- **I²C versus one-wire.** Unlike the DHT11, the BMP280 never returned a bad frame: I²C is clocked by the master, so the operating system’s timing jitter cannot corrupt the data, and the hardware peripheral does the bit-level work.

5 Conclusion

The BMP280 was read successfully from both a Raspberry Pi and an ESP32 over the I²C bus, giving calibrated pressure and temperature and a derived altitude. The Raspberry Pi printed a clean, steady stream of readings — 929.3 hPa, about 719 m pressure altitude, temperature settling from 31.4 to 30.1 °C — and the ESP32 delivered the same three quantities as a live web dashboard on its own Wi-Fi network, at 940.2 hPa and 627 m on a different day; every altitude value reproduced the barometric formula applied to its pressure.

The experiment made two points concrete: a barometric altitude is a pressure measurement in disguise, and depends on the sea-level reference as much as on height; and a hardware bus such as I²C is a far more robust way to attach a sensor than the timing-critical one-wire link of the previous experiment. The ESP32 part also took the first step from reading a sensor to *publishing* its data over a network — the essence of an IoT node.